

Introduction to Python Programming

SIMCODES 2026

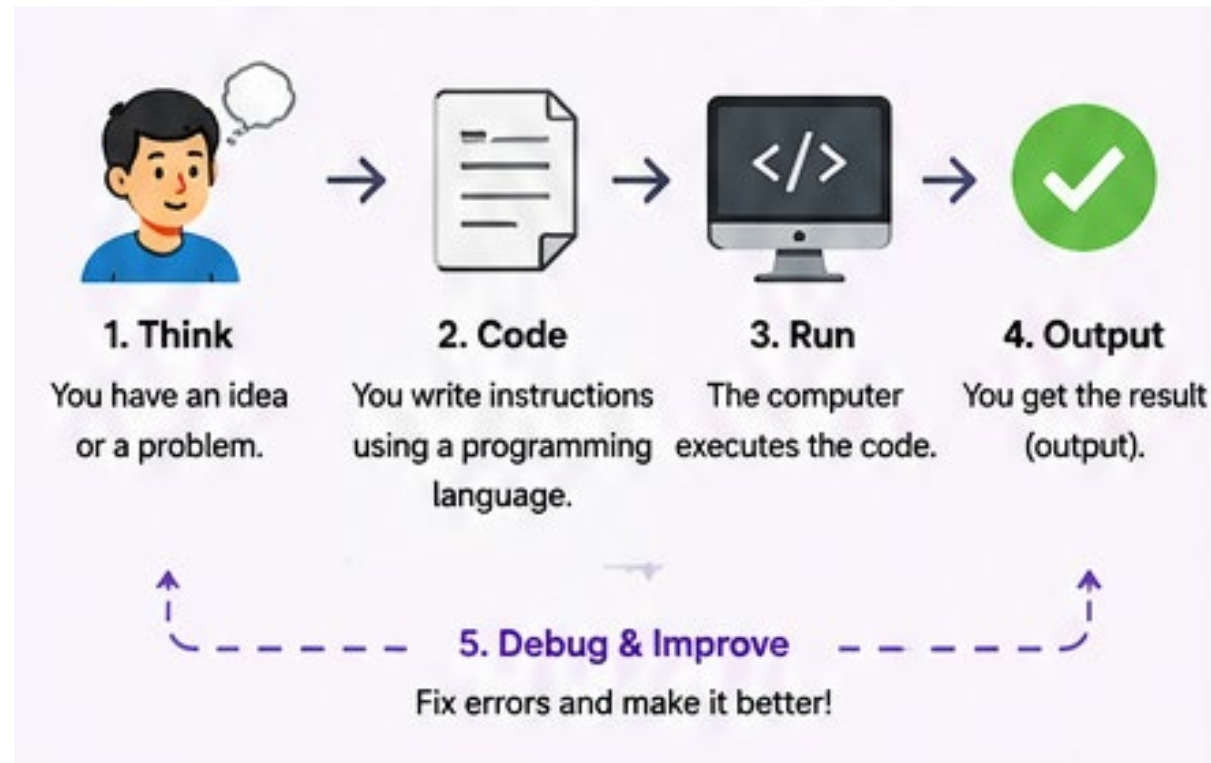
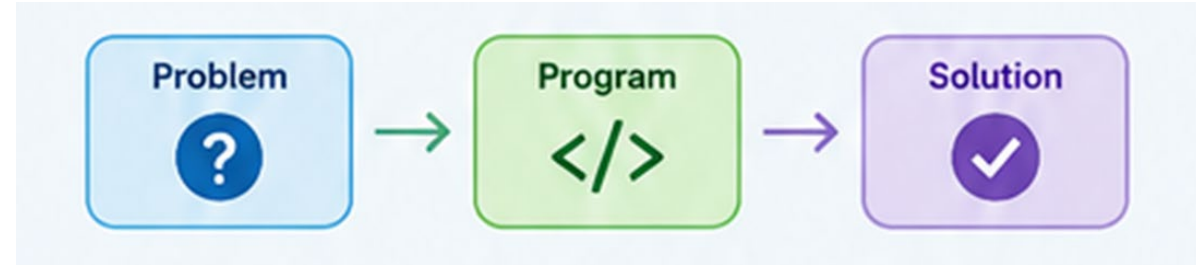
Iowa State University | Ames National Lab
Dulitha Prasanna

06/03/2026



What is a programming language?

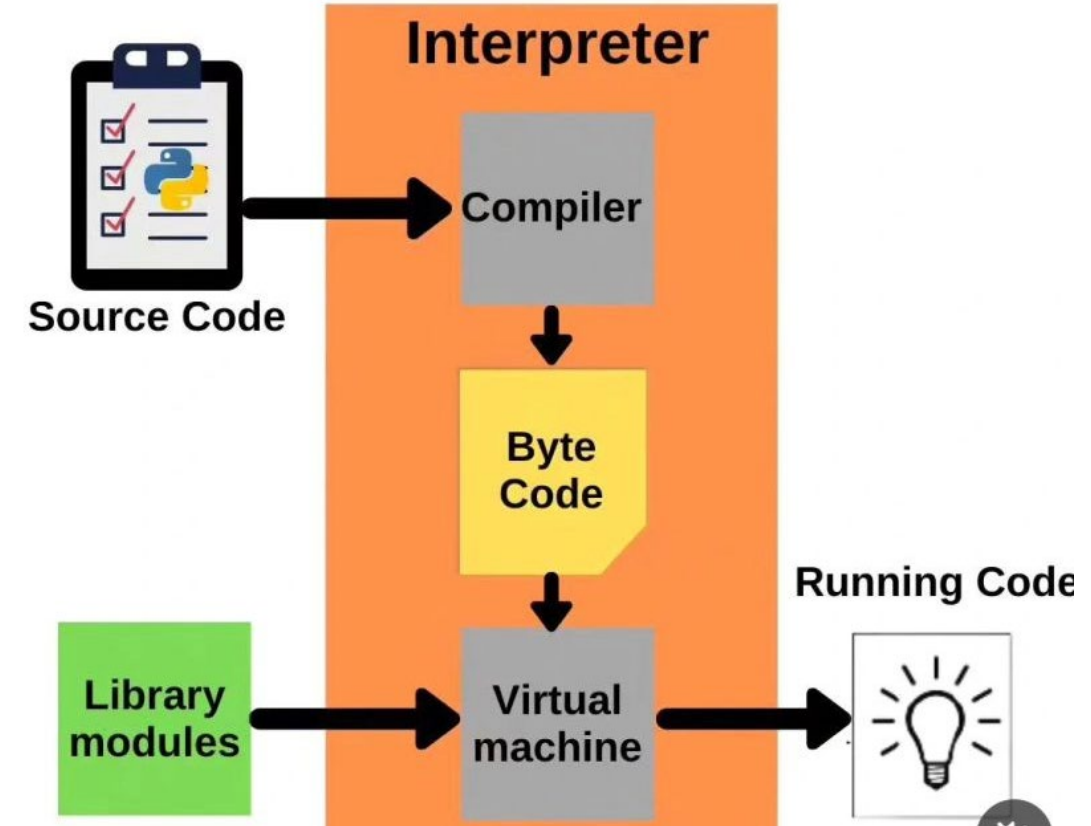
- ❑ Give a computer **step-by-step instructions** (a program) to solve a problem or a specific task.
- ❑ Is a sort of **rules and symbols** that allow us to communicate with computers.
 - ❑ Human Languages - English, French ...
 - ❑ Machine Languages – 0's and 1's
 - ❑ Programming Languages – Python, Java, C ...
- ❑ Act as an intermediary between humans and computers, **translating ideas into instructions** that computers can understand.



Python Programming



Python scripts are **plain-text files** executed from top to bottom, whereas **Jupyter Notebooks** are interactive scripts split into **executable cells**.



First Program



- ❑ Create hello.py to run the script, `python hello.py` in the terminal
- ❑ Create hello.ipynb and choose the venv Python kernel and run the cell.

Variables and Data Types

❑ What is a Variable?

A variable **stores information** (data) in memory that can be reused.

❑ Data types define what kind of information the variable contains.

❑ Python automatically determines the type based on the assigned value. **Dynamic typing**

```
age = 21
pi = 3.14
molecule = "Ethanol"
is_valid = True
```

```
print(age)
print(pi)
print(molecule)
print(is_valid)
```

```
print(type(age))
```

Data Type	Example	Description
Integer (int)	10	Whole numbers
Float (float)	36.5	Decimal numbers
String (str)	"Hello"	Text
Boolean (bool)	True	True/False values

Chemistry Problem - Calculate Gibbs Free Energy

$$\Delta G^0 = \Delta H^0 - T\Delta S^0$$

```
deltaH = -541.5  #kJ/mole  
deltaS = 10.4    #kJ/(mole K)  
temp = 298      #Kelvin  
deltaG = deltaH - temp * deltaS
```

```
deltaH, deltaS, temp = -541.5, 10.4, 298  
deltaG = deltaH - temp * deltaS  
print(deltaG)
```

Task

Assign variables and calculate ΔG .

Given:

$\Delta H = -541.5 \text{ kJ/mol}$

$T = 298 \text{ K}$

$\Delta S = 10.4 \text{ kJ/mol}\cdot\text{K}$

Write a Python program to print ΔG .

Containers

❑ Store many values in a single variable.

```
temp1 = 20
temp2 = 25
temp3 = 30
temp4 = 35
```



```
temperatures = [20, 25, 30, 35]
# One variable stores all values!
```



List

Ordered,
changeable



Tuple

Ordered,
unchangeable



Dictionary

Key-value
pairs



Set

Unordered,
unique items

1. LIST

Ordered, changeable

A list stores items in a particular order.
Items can be modified, added, or removed.

```
temperatures = [20, 30, 40, 50]
print(temperatures[0]) # 20

temperatures.append(60)
print(temperatures) # [20, 30, 40, 50, 60]
```

2. TUPLE

Ordered, unchangeable

Similar to a list, but items cannot be
modified after creation.

```
coordinates = (1.2, 3.4, 5.6)
print(coordinates[0]) # 1.2

# coordinates[0] = 10 ❌ (Not allowed)
```

3. DICTIONARY

Key-value pairs

Stores data as key-value pairs.
Keys must be unique.

```
atomic_weights = {
    "H": 1.008,
    "C": 12.01,
    "O": 16.00
}
print(atomic_weights["O"]) # 16.00
```

4. SET

Unordered, unique items

Stores only unique items.
Order is not guaranteed.

```
elements = {"H", "C", "O", "O"}
print(elements) # {'C', 'O', 'H'}

elements.add("N")
print(elements) # {'C', 'O', 'H', 'N'}
```

List Methods and Slice

	Description	Code example	Diagram
append()	Adds an element to the end of the list	letters = ['a', 'b', 'c'] letters.append('d')	
extend()	Adds the elements of a list to the end of another list	letters = ['a', 'b', 'c'] letters.extend(['d', 'e', 'f'])	
insert()	Adds an element at a specific index	letters = ['a', 'b', 'c'] letters.insert(1, 'x')	
remove()	Removes the first occurrence of an element	letters = ['a', 'b', 'c', 'b'] letters.remove('b')	
pop()	Removes and returns the element at a given index	letters = ['a', 'b', 'c', 'd'] letter = letters.pop(1)	
count()	Returns the number of times a value appears in a list	letters = ['c', 'b', 'c', 'c', 'd'] print(letters.count('c'))	
sort() in @NikkiSiapno	Sorts the list in ascending order	letters = ['c', 'a', 'd', 'a', 'b'] letters.sort()	
reverse() in @ChrisStaud	Reverses the order of elements in the list	letters = ['a', 'b', 'c'] letters.reverse()	

This is a list

```
energy_kcal = [-13.4, -2.7, 5.4, 42.1]
```

I can determine its length

```
energy_length = len(energy_kcal)
```

print the list length

```
print('The length of this list is', energy_length)
```

```
print(energy_kcal[0])
```

```
energy_kilojoules = energy_kcal[1] * 4.184
```

```
print(energy_kilojoules)
```

Slicing a list

```
new_list = list_name[start:end]
```

```
short_list = energy_kcal[0:2]
```

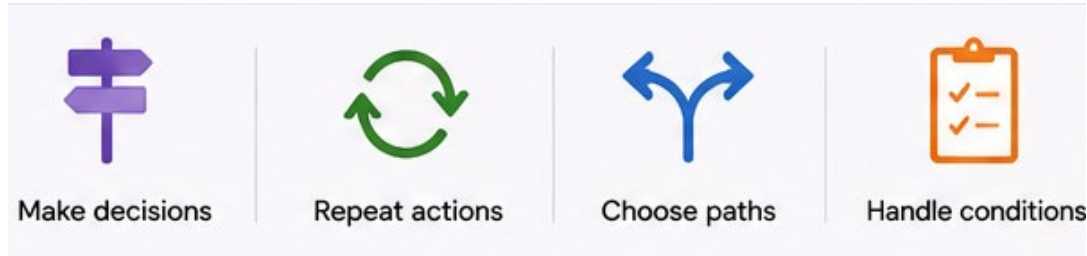
```
slice1 = energy_kcal[1:]
```

```
slice2 = energy_kcal[:3]
```

```
print('slice1 is', slice1)
```

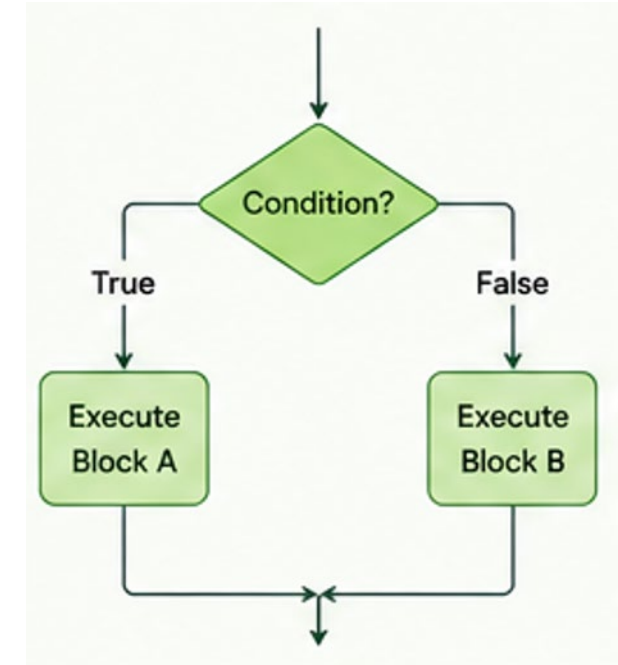
```
print('slice2 is', slice2)
```

Control Flow Statements



Python Control flow is mainly divided into:

- ❑ Selection (Decision Making) - if, elif, else
- ❑ Iteration (Repetition) - for loop, while loop
- ❑ Jump Statements - break, continue, pass



```

deltaG = -1.0
if deltaG < 0 :
    print('Spontaneous')
elif deltaG > 0:
    print('non-spontaneous')
else:
    print('equilibrium')
  
```

Comparison Operators	
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
==	Equal to
!=	Not equal to

Loops

```
temperatures = [20, 30, 40, 50, 60]
```

```
print(temperatures[0])  
print(temperatures[1])  
print(temperatures[2])  
print(temperatures[3])  
print(temperatures[4])
```

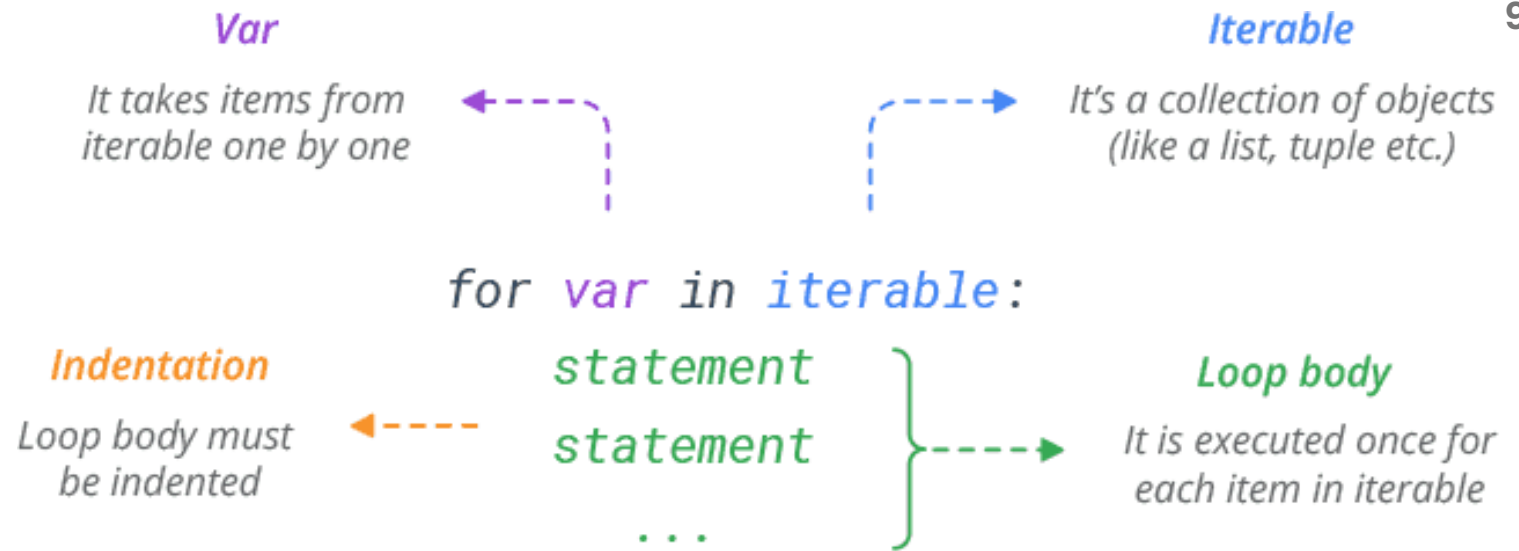
```
for temperature in temperatures:  
    print(temperature)
```

```
range(start, stop, step)  
print(range(5))
```

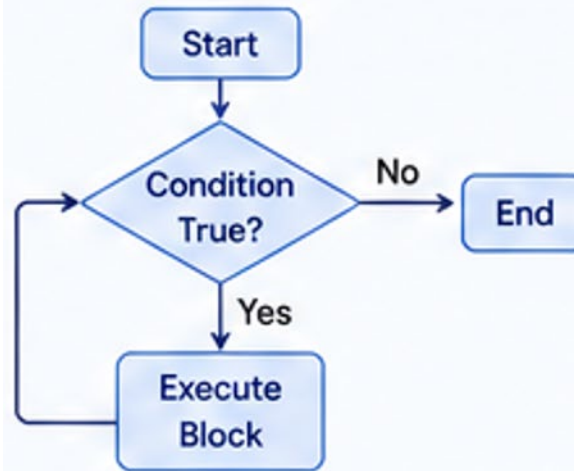
```
for i in range(1, 6):  
    print(i)
```

```
for i in range(1, 6):  
    print(i)
```

```
for i in range(0, 11, 2):  
    print(i)
```



WHILE LOOP (repeat while condition is True)

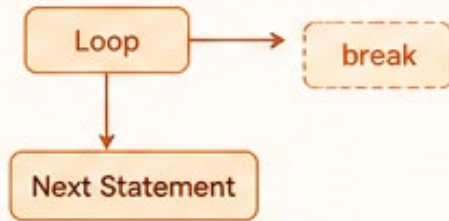


Example:

```
i = 1  
while i <= 5:  
    print(i)  
    i = i + 1
```

Jump Statements

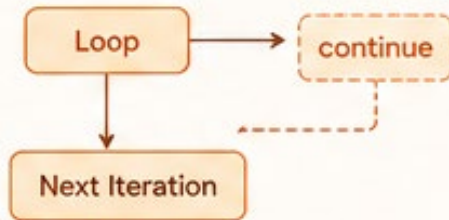
BREAK – exit the loop completely



Example:

```
for i in range(5):
    if i == 3:
        break
    print(i)  # 0 1 2
```

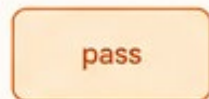
CONTINUE – skip the current iteration



Example:

```
for i in range(5):
    if i == 2:
        continue
    print(i)  # 0 1 3 4
```

PASS – do nothing (placeholder)



Example:

```
if True:
    pass  # placeholder
print("Done")
```

Task

Select a catalyst that provides a safe and efficient reaction rate at room temperature (298 K). You will calculate the rate constant (k) using the Arrhenius equation:

$$k = Ae^{-\frac{E_a}{RT}}$$

k is the rate constant (s^{-1})

A is the pre-exponential factor (assume $1.0 \times 10^{13} s^{-1}$)

E_a is the activation energy in J/mol

R is the ideal gas constant ($8.314 J/(mol K)$)

T is the temperature in Kelvin (298 K)

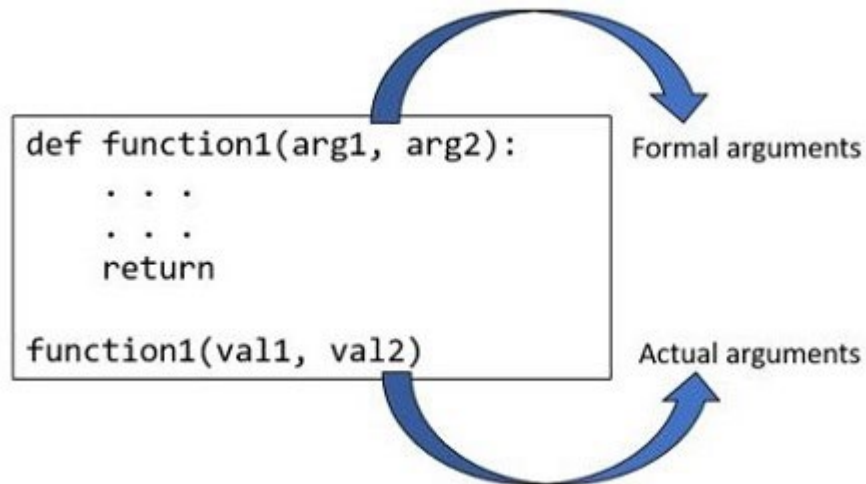
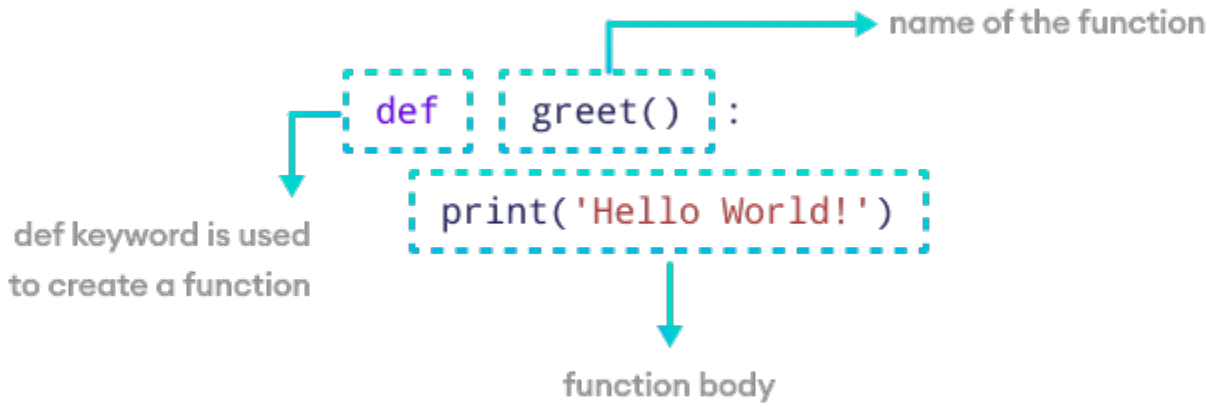
You have a dictionary containing 5 different E_a for catalysts. $E_a = [85000, 75000, 50000, 40000, 30000]$

Write a script that **loops through the dictionary and calculates the rate constant k** for each catalyst using the `math.exp()` function (`import math`). If **k is less than 0.01**, the reaction is too slow, Print a message and skip to the next catalyst (`continue`).

If **k is greater than 1000**, the reaction will generate too much heat and explode! Print an emergency warning and shut down the entire screening process immediately (`break`).

If **k is in the safe operating range**, print out that you have found a viable catalyst.

Functions



Adding a docstring to a function, a block quote directly underneath the function definition.

$$\Delta G^0 = \Delta H^0 - T\Delta S^0$$

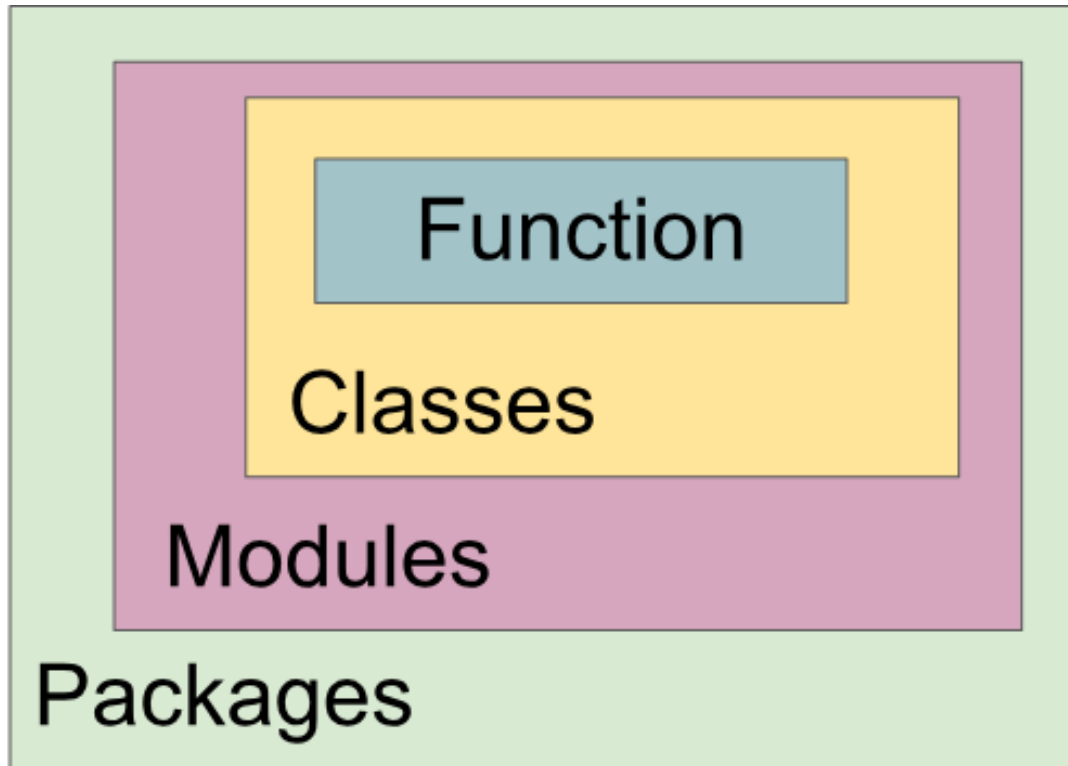
Task

- Write a function to calculate ΔG .
- Extend this function to print whether the reaction is spontaneous or not.
- Evaluate this function for $\Delta H = -250 \text{ kJ/mol}$ $\Delta S = 10.0 \text{ kJ/mol}\cdot\text{K}$ and $\Delta H = 250 \text{ kJ/mol}$ $\Delta S = 10.0 \text{ kJ/mol}\cdot\text{K}$ At $T = 298 \text{ K}$.

```
def example_function(param1, param2):
    """
    This is an example of a docstring
    for a Python function.

    Docstrings are enclosed in triple
    quotes and appear immediately
    after the function definition.
    """
    # Function implementation
```

Scientific Software Design with Python



- ❑ Python software has three flavors: **modules**, **packages**, and **libraries**.
- ❑ A module is a single *.py file.
- ❑ Modules are building blocks of packages and libraries.
- ❑ All the actual effort goes into writing modules.
- ❑ **Packages are modules** distributed together.
- ❑ Name of the directory will be the name of the package.
- ❑ The **__init__.py file** is usually empty, but is essential because that's how Python determines your directory is a package.
- ❑ The ***.py files** are plain text and are where you put the code.

Python Package for Reaction Analysis

reaction_analysis/

```

├── reaction_analysis/
│   ├── __init__.py
│   ├── thermodynamics.py
│   └── main.py
├── tests/
│   └── test_thermodynamics.py
└── README.md
  
```

thermodynamics.py

- ☐ Function for reaction type (exothermic or endothermic)
- ☐ Function for delta G calculation
- ☐ Function for spontaneity
- ☐ Function for equilibrium Constant

$$K = \exp\left(\frac{-\Delta G^\circ}{RT}\right)$$

$$\Delta G^\circ = \Delta H^\circ - T\Delta S^\circ$$

main.py

```
from reaction_analysis.thermodynamics import ...
```

```
# Reaction parameters
```

```

delta_H = -100    # kJ/mol
delta_S = -0.2    # kJ/mol/K
T = 298           # K
R = 0.008314     # kJ /mol/K
  
```

```
# Calculations
```

```

delta_G = calculate_delta_g(
    delta_H,
    delta_S,
    T
)
K_eq = calculate_equilibrium_constant(
    delta_G,
    T
)
reaction_type = classify_reaction(
    delta_G
)
  
```

Tests

- ❑ pip install pytest
- ❑ To run in the terminal -> pytest
- ❑ assert actual == expected

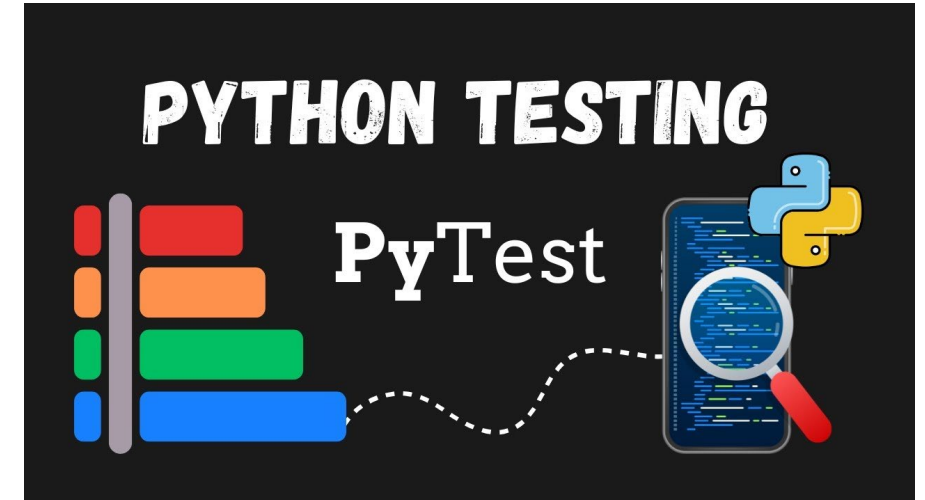
test_thermodynamics.py

```
from reaction_analysis.thermodynamics import  
calculate_delta_g
```

```
def test_delta_g():
```

```
    result = calculate_delta_g(  
        -100,  
        -0.2,  
        298  
    )
```

```
    assert result == -40.4
```



Python Classes



A class is a **blueprint** for creating objects.

- ✓ Stores data (attributes)
- ✓ Has behavior (methods)
- ✓ Creates objects

The Idea



Class
(blueprint)



Objects
(real instances)

Example: Reaction Class

```
class Reaction:
    def __init__(self, dh, ds, T):
        self.dh = dh    # attribute
        self.ds = ds    # attribute
        self.T = T      # attribute
    def calculate_delta_g(self):
        return self.dh - self.T * self.ds
```

Create Objects

```
rxn1 = Reaction(-100, -0.2, 298)
rxn2 = Reaction(-50, 0.1, 350)
```



Each object
has its own
data!

Use Methods (Behavior)

```
dg1 = rxn1.calculate_delta_g()
dg2 = rxn2.calculate_delta_g()
```

Outputs

```
dg1 = -40.40 kJ/mol
dg2 = -85.00 kJ/mol
```

Same method,
different results!

Key OOP Concepts



Class

Blueprint / template



Object

Instance of a class



Attribute

Data stored in the object (dh, ds, T)



Method

Function that belongs to the object



Thank You!

dulithaprasanna@gmail.com
dulitha@iastate.edu